

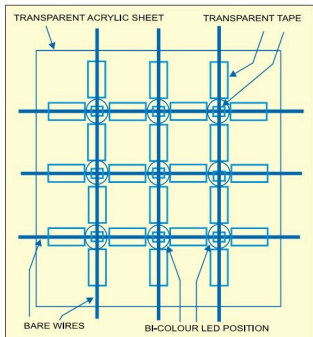


Fig. 5: Touchpad placement on the LEDs

PARTS LIST

Semiconductors:

Arduino
LED1-LED9 - Arduino Duemilanove board
- Common-cathode bi-colour LEDs

Resistors (all 1/4-watt, 45% carbon):

R1-R6 - 220-ohm
R7-R12 - 100-kilo-ohm

Miscellaneous:

S1 - On/Off SPDT switch
- A bunch of header pins
- A bunch of electrical wires
- A small square sheet of transparent acrylic, ~1mm thick, 8 cm on the side
- Transparent cello tape

ming environment. Arduino IDE is available for Windows, MAC and Linux systems.

Open Source Arduino IDE can be freely downloaded from www.arduino.cc/ website. For a step-by-step guide on getting started with the Arduino and setting up the board with your computer, visit <http://arduino.cc/en/Guide/HomePage>

Sensing the touchpad inputs. To sense the touch inputs, it needs to be determined which intersection point was pressed. In software, the following method is used to achieve this:

Step 1. Set all the column lines to output low.

Step 2. Set the row lines as inputs with the internal pull-ups activated.

Step 3. Take an analogue read on each row line until the value drops below a given threshold to find out in which row the input is pressed.

Step 4. Repeat steps 1 through 3, but now with the columns as inputs and the rows as outputs to find out in which column the input is pressed.

To minimise the effects of noise, a number of readings are taken and then averaged in the program. The averaged result is then compared against the threshold.

Addressing and lighting the LED matrix. It is also required to store the status of the LEDs in the software, i.e., information about whether an LED is on or off, and if it is on then which colour – red or green. For this purpose, two nine-cell arrays are used, i.e., two sets of nine bits are stored. The first set stores the on/off status of the nine bi-colour LEDs, while the second set stores the colour of the LEDs. For example, if '1' is used to represent the 'on' state and '0' for the 'off' state of the LED, the first nine-cell array (or the 3x3 LED matrix) looks like:

1. For LED7 to light up

```
000
000
100
```

2. For diagonal LEDs (LED1, LED5 and LED9) to light up

```
100
010
001
```

If the LED cells are enumerated

starting from bottom left, the LED status can be written as a series of bits. For example, when only LED7 is lit up, the bits 10000000 (binary equivalent 256) represent this condition. Similarly, when only the diagonal LEDs are lit up, the bits 001010100 (binary equivalent 84) represent this state. Each series of nine bits can be represented in the abridged form as a binary number. So instead of using an array to store the state of the matrix, you can use just one number.

You can similarly represent the colour of the LEDs by binary numbers. If '1' represents red and '0' represents green, the following matrix represents an alternating colour pattern:

010 = green, red, green

101 = red, green, red

010 = green, red, green

The following algorithm is used for lighting up the LED matrix:

Step 1. Do a bit-wise addition of the on/off state with a binary 1 (i.e., bit-masking).

Step 2. If it is true, the LED is lit. Now do a bit-wise addition of the colour state with a binary 1.

Step 3. If it is true, the red LED lights up. If it is false, the green LED lights up.

Step 4. Shift both the on/off state and colour state one bit to the right (i.e., bit-shifting).

Step 5. Repeat steps 1 through 4 until all nine bits have been read.

Note that the matrix is filled backwards starting with cell nine and then proceeding to cell one.

Also, the on/off and colour states are stored as an unsigned integer-type (word) instead of a signed integer-type. This is because if you are not careful in bit-shifting, the sign of the variable can get inadvertently changed.

Construction

You need one perfboard (or strip board), nine common-cathode bi-colour LEDs, six 220-ohm resistors, six 100-kilo-ohm resistors, one SPDT switch, a bunch of header pins, a